Robotic Systems 1

Project 1 – Dobot Magician Lite – Maze Solving

Nikoletta Biri
School of Manufacturing Systems and Networks
Arizona State University
ASU ID: 1240137928
nbiri@asu.edu

***Abstract*— This report presents the first project of RAS 545 – Robotic Systems I course. It describes a vision-based maze solving system using the Dobot Magician Lite robot. The system integrates camera calibration, image processing, path planning, and robot control to autonomously solve a maze. A captured maze image is processed to detect walls and paths, and a Breadth-First Search (BFS) algorithm computes the shortest path. The solution is converted from pixel to robot coordinates using an affine transformation and executed by the robot, demonstrating effective integration of perception, planning, and control.**

Keywords*— *Dobot Magician Lite, Maze Solving, Computer Vision, Camera Calibration, Image Processing, Breadth-First Search (BFS), Path Planning, Robot Control, Affine Transformation

Video Link:

https://youtube.com/shorts/gi8C1Oyw_Mk?si=D7Ah9MhOHElXubpp

I. Introduction

The Dobot Magician Lite is an educational robotic arm capable of performing precise motion tasks within a defined workspace. In this project, the robot is used to solve a maze using a combination of computer vision and motion control techniques.

The task involves capturing an image of a maze using a camera, processing the image to detect maze structure, solving the maze algorithmically, and commanding the robot to trace the solution path. This project highlights the integration of perception, coordinate transformation, and motion execution, which are fundamental components in robotics applications.

A. Hardware Setup

The Dobot Magician Lite robot was connected to a laptop via USB and powered using an external power supply. A camera was also connected to the laptop and positioned above the workspace to capture the maze. A physical maze was placed within the camera's field of view for processing. The robot workspace was aligned with the camera view to ensure consistent coordinate mapping.

B. Software Setup

Python (Version 3.14) was used as the primary programming language, with Visual Studio Code (Version 1.109.0) as the development environment. Multiple provided scripts were utilized for camera input, coordinate extraction, transformation, and robot control. Key files included camera utilities, robot utilities, and maze processing scripts.

C. Programming Logic

The program was implemented in Python to integrate camera input, image processing, path planning, and robot motion control for solving the maze. The overall image processing pipeline used to extract the maze structure is shown in Fig. 1.

First, the maze image was captured using a camera positioned above the workspace. The boundaries were identified by selecting the four corner points and applying a perspective transformation to obtain a top-down view, simplifying the maze into a structured grid.

Next, image processing techniques were used to detect circular markers and grid cells. Each cell was classified as either a wall or a free path, generating a binary grid representation where 0 indicates a wall and 1 indicates a valid path.

The maze was then solved using a Breadth-First Search (BFS) algorithm to compute the shortest path between the start and goal positions. The resulting path was stored as a sequence of grid points.

The path was mapped back to the original image to obtain pixel coordinates, which were converted into robot coordinates using an affine transformation matrix from camera calibration.

Finally, the Dobot Magician Lite robot was commanded to follow the path using point-to-point motion, executing the trajectory through each transformed coordinate.

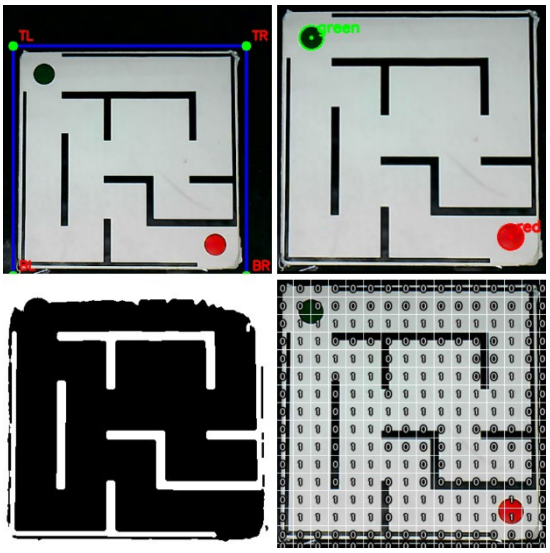

Fig. 1. Maze processing pipeline includes detected maze corners used for perspective transformation, detection of start and goal markers, binary image for wall segmentation, and grid-based representation used for path planning.

II. Results

The proposed system successfully solved the maze and executed the computed path using the Dobot Magician Lite robot. The image processing pipeline accurately detected the maze boundaries, start and goal markers, and generated a valid grid representation of the maze.

The Breadth-First Search (BFS) algorithm computed a feasible shortest path between the start and goal positions. The resulting path was mapped from grid coordinates to pixel coordinates and then converted into robot coordinates using the affine transformation matrix obtained during camera calibration.

The robot followed the computed trajectory using point-to-point motion and was able to trace the solution path with good accuracy. Minor deviations were observed due to calibration errors and mechanical tolerances; however, the robot consistently reached the goal position.

The final solution path generated by the algorithm and executed by the robot is shown in Fig. 2, demonstrating the effectiveness of the integrated perception, planning, and control system.

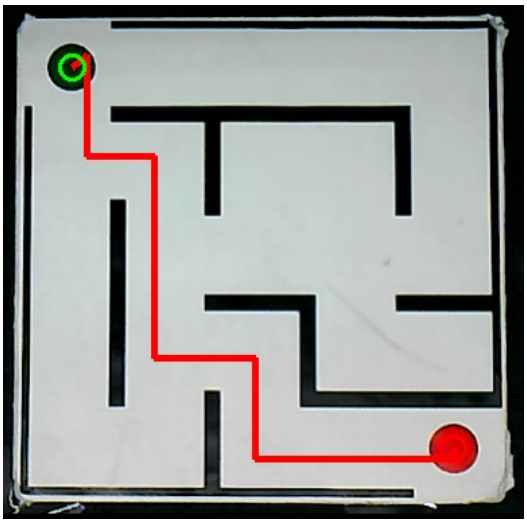

Fig. 2. Final maze solution showing the shortest path computed using BFS and executed by the robot.

III. Discussion

During the implementation, several practical challenges were encountered related to both software and hardware integration. The provided Python scripts required modification, as certain sections were left as “TO DO” tasks. These parts had to be completed and adapted to ensure proper functionality of the overall pipeline.

One of the main difficulties involved color detection. Initially, the system failed to correctly detect the green marker representing the start position. This issue was resolved by tuning the HSV color thresholds, specifically adjusting the upper and lower bounds for the green range until reliable detection was achieved under the given lighting conditions.

Another challenge was related to robot reachability. The robot was unable to reach one corner of the maze due to workspace limitations. As a result, the pixel-to-robot coordinate mapping had to be recalibrated by re-recording the corresponding pixel and robot coordinates within a reachable region. This ensured that all planned path points could be executed by the robot without exceeding its physical limits.

Despite these challenges, the system was successfully implemented after iterative adjustments. These issues highlight the importance of calibration, parameter tuning, and consideration of hardware constraints when integrating perception and motion in robotic systems. These challenges emphasize the importance of iterative tuning in real-world robotic systems.

IV. Conclusion

In this project, a vision-based maze solving system was successfully implemented using the Dobot Magician Lite robot. The system integrated camera-based perception, image processing, path planning, and robot motion control to autonomously solve a maze.

The image processing pipeline enabled reliable detection of maze features, while the Breadth-First Search (BFS) algorithm efficiently computed a valid path between the start and goal positions. The use of an affine transformation allowed accurate mapping from pixel coordinates to robot coordinates, enabling the robot to execute the planned trajectory.

Despite challenges related to color detection and robot reachability, the system was successfully calibrated and adjusted to achieve consistent performance. The results demonstrate the effectiveness of combining perception, planning, and control in a robotic application and provide a strong foundation for more advanced autonomous systems.

References

[1] Dobot Magician Lite, “K12 AI & Robotics Education Solution.” [Online]. Available: DOBOT Magician Lite | K12 AI & Robotics Education Solution

[2] Prof. Mostafa Yourdkhani, “Maze Solving Project Instructions”, Robotics Systems I, ASU, 2026

[3] Student Submission, N. Biri, “RAS 545 - Robotics Systems I - Project 1 - Maze Solver, Youtube. [Online]. Available: https://youtube.com/shorts/gi8C1Oyw_Mk?si=D7Ah9MhOHElXubpp